

XFUEL: Zero-Knowledge Bridge for Cross-Chain Yield Automation

Technical Whitepaper v2.0
December 2025

Abstract

XFUEL is a zero-knowledge bridge protocol that enables trustless, non-custodial cross-chain asset transfers between Theta Network and Cosmos ecosystem, with automated yield optimization powered by liquid staking tokens (LSTs). By leveraging ZK-SNARK proofs for transaction validation and IBC (Inter-Blockchain Communication) protocol for cross-chain messaging, XFUEL achieves sub-4-second finality for TFUEL → ibcTFUEL → LST swaps while maintaining cryptographic security guarantees.

This whitepaper presents the technical architecture, cryptographic primitives, tokenomics, risk analysis, and mitigation strategies for the world's first ZK-enabled perpetual yield bridge — a self-evolving cross-chain economy where protocol revenue drives deflation, real yield distribution, and decentralized governance.

Key Metrics (v1.0.0):

- **Bridge Finality:** < 4 seconds (ZK proof generation + verification)
 - **Target APY:** 30-38% (Cosmos LSTs: stkTIA, stkATOM, stkXPRT)
 - **Security Model:** Zero-knowledge proofs + IBC light client verification
 - **Liquidity Model:** Concentrated liquidity pools with impermanent loss backstop
-

Table of Contents

1. Introduction & Vision
2. The Opportunity
3. Technical Architecture
 - 3.1 Non-Connect Deposit Flow
 - 3.2 Zero-Knowledge Proof System
 - 3.3 IBC Integration & ibcTFUEL Minting
 - 3.4 Smart Contract Layer
 - 3.5 Yield Optimization Engine
4. ibcTFUEL Tokenomics
5. XF Governance Token Tokenomics

- 6. Security Model & Cryptographic Guarantees
- 7. Risks & Mitigations
- 8. Governance & Sustainability
- 9. Roadmap
- 10. Conclusion
- 11. References & Appendices

1. Introduction & Vision

1.1 The Cross-Chain Yield Gap

Theta Network, powered by EdgeCloud and the TDROP 2.0 AI agent economy, generates substantial TFUEL earnings for node operators. Meanwhile, the Cosmos ecosystem offers liquid staking tokens (LSTs) with yields ranging from 30-38% APY. These two ecosystems remain siloed, with no secure, trustless bridge enabling Theta holders to access Cosmos yield opportunities.

Traditional bridges suffer from:

- **Custodial risk:** Multisig vulnerabilities, admin key compromises
- **Long settlement times:** 10-60 minutes for cross-chain confirmations
- **Complex UX:** Requires multiple wallets, browser extensions, and manual coordination
- **Lack of privacy:** All transaction details exposed on-chain

1.2 XFUEL's Solution: ZK Bridge + Automated Yield

XFUEL solves these problems by introducing:

1. **Zero-Knowledge Proofs (ZK-SNARKs):** Cryptographically prove transaction validity without revealing sensitive details. Users deposit TFUEL on Theta, and a ZK proof is generated confirming the deposit's authenticity. This proof is verified on Persistence (Cosmos) in constant time, enabling instant minting of ibcTFUEL.
2. **Non-Connect Deposits:** Users send TFUEL via QR code or address copy-paste — no wallet extensions, no WalletConnect, no browser dependencies. The backend listener detects deposits, generates ZK proofs, and triggers cross-chain routing automatically.
3. **IBC Protocol Integration:** Once the ZK proof is verified, ibcTFUEL is minted 1:1 with deposited TFUEL and transferred via IBC channel-190 to Persistence, where it can be swapped for LSTs (stkTIA, stkATOM, pSTAKE BTC) on Dexter DEX or staked via pStake.
4. **Automated Yield Optimization:** Smart contracts automatically route ibcTFUEL to the highest-yielding LST strategy, rebalancing weekly based on oracle-fed APY data.

1.3 Vision: The Perpetual Yield Pumping Station

XFUEL is more than a bridge — it's a living economic system where:

- **Protocol revenue** (swap fees, yield cuts, lottery rake) is split 90/10 between veXF holders and the Innovation Treasury
- **Buyback & burn** mechanisms drive XF deflation (25% of revenue)
- **Revenue-backed receipts (rXF)** are minted from 15% of revenue, offering 4× governance voting power when locked for 365 days
- **Innovation Treasury** funds new experiments, protocol acquisitions, and spin-out tokens (50% airdropped to veXF/rXF holders)

2. The Opportunity

2.1 Theta EdgeCloud & TDROP 2.0 Growth

TDROP 2.0 (launched December 17, 2025) shifts Theta's incentive model toward decentralized AI agents and compute workloads. Edge Node operators now earn TFUEL by providing:

- Video transcoding
- AI model inference
- Distributed storage
- Real-time rendering

This creates a growing pool of TFUEL holders seeking yield opportunities beyond simple hodling.

2.2 Cosmos LST Ecosystem

The Cosmos ecosystem has matured into a liquid staking powerhouse:

Asset	Provider	APY	Liquidity	Risk Profile
stkTIA	Stride	38.2%	\$45M	Medium (Celestia dependency)
stkATOM	Stride	32.5%	\$120M	Low (ATOM is hub token)
stkXPRT	pStake	30.1%	\$8M	Medium (smaller cap)
pSTAKE BTC	pStake	28.0%	\$15M	High (wrapped BTC risk)

These LSTs offer:

- **Instant liquidity:** Tradeable on DEXs (Osmosis, Dexter, Crescent)
- **Auto-compounding:** Staking rewards auto-added to principal
- **Governance participation:** Vote on Cosmos Hub proposals while earning yield

2.3 Market Opportunity

- **Theta Edge Node operators:** ~10,000+ nodes generating ~\$5M TFUEL monthly
- **Target capture rate:** 5-10% in Year 1 → \$250K-\$500K monthly volume

- **Protocol revenue:** 0.3% swap fee → \$750-\$1,500/month initially, scaling with adoption
-

3. Technical Architecture

3.1 Non-Connect Deposit Flow

XFUEL eliminates the need for WalletConnect or browser extensions by implementing a **manual deposit flow**:

User Journey

1. **User selects LST target** (e.g., stkTIA) in the XFUEL web/mobile app
2. **App displays deposit address + QR code** for the TFUEL deposit address
3. **User opens Theta Wallet**, scans QR code, and sends TFUEL
4. **Backend listener detects deposit** on Theta blockchain (within 6 seconds)
5. **ZK proof is generated** confirming deposit validity
6. **Proof is verified on Persistence**, ibcTFUEL is minted and transferred via IBC
7. **LST is minted and sent** to user's Cosmos address (extracted from deposit memo)

Technical Components





Code Reference: Theta Listener

The backend listener continuously scans the Theta blockchain for deposits:

File: `backend/ibc/listener.ts`

Key functions:

- `initializeListener()` : Connects to Theta RPC, loads last processed block
- `startListener()` : Infinite polling loop
- `scanBlocksForDeposits()` : Iterates through blocks, identifies TFUEL transfers to deposit address
- `handleDeposit()` : Extracts recipient address from transaction data, waits for confirmations, triggers routing

Deposit Detection Logic:

```
// Simplified from backend/ibc/listener.ts (lines 117-149)
async function scanBlocksForDeposits(fromBlock: number, toBlock: number) {
  const deposits = []
  const depositAddress = IBC_CONFIG.theta.depositAddress.toLowerCase()

  for (let blockNum = fromBlock; blockNum <= toBlock; blockNum++) {
    const block = await provider.getBlock(blockNum, true)

    for (const tx of block.prefetchedTransactions) {
      if (tx.to?.toLowerCase() === depositAddress && tx.value > 0n) {
        deposits.push({
          txHash: tx.hash,
          from: tx.from,
          to: tx.to,
          value: tx.value.toString(),
          blockNumber: blockNum,
          timestamp: block.timestamp
        })
      }
    }
  }

  return deposits
}
```

Recipient Address Extraction:

Users encode their Cosmos address (e.g., `persistence1abc...`) in the transaction's data field as hex-encoded UTF-8:

```
// Simplified from backend/ibc/listener.ts (lines 224-245)
function decodeRecipientAddress(data: string): string {
  const hex = data.startsWith('0x') ? data.slice(2) : data
  const decoded = Buffer.from(hex, 'hex').toString('utf8')

  // Validate Persistence address format
  if (decoded.startsWith('persistence1') && decoded.length === 45) {
    return decoded
  }

  return '' // Manual processing required if invalid
}
```

3.2 Zero-Knowledge Proof System

XFUEL uses **ZK-SNARKs** (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) to prove deposit validity without revealing transaction details to the Persistence chain.

Why ZK Proofs?

1. **Privacy:** Transaction amounts and sender addresses remain private
2. **Efficiency:** Verification is constant time (~50ms) regardless of proof complexity
3. **Security:** Cryptographic guarantee that deposit occurred on Theta without trusting relayers

ZK Circuit: ThetaDepositVerifier

The ZK circuit (written in Circom) takes private inputs and generates a proof that the deposit is valid:

Private Inputs:

- `txHash`: Theta transaction hash (32 bytes)
- `blockNumber`: Block number of deposit (uint256)
- `amount`: TFUEL amount in wei (uint256)
- `sender`: Sender address on Theta (20 bytes)
- `merkleProof`: Merkle proof of transaction inclusion in block

Public Signals:

- `depositCommitment`: Hash of (txHash, blockNumber, amount)
- `recipientHash`: Hash of recipient Cosmos address
- `nonce`: Unique nonce to prevent replay attacks

Circuit Logic:

```
// Simplified ZK circuit (conceptual)
template ThetaDepositVerifier() {
  // Private inputs
  signal private input txHash[32];
  signal private input blockNumber;
  signal private input amount;
  signal private input sender[20];
  signal private input merkleProof[10][32]; // 10-depth Merkle tree
  signal private input merkleRoot[32];

  // Public outputs
  signal output depositCommitment;
  signal output recipientHash;
  signal output nonce;

  // 1. Verify Merkle proof (transaction is in block)
  component merkleVerifier = MerkleTreeVerifier(10);
  merkleVerifier.leaf <== Poseidon(txHash);
  merkleVerifier.root <== merkleRoot;
  for (var i = 0; i < 10; i++) {
    merkleVerifier.pathElements[i] <== merkleProof[i];
  }
  merkleVerifier.valid === 1; // Assert proof is valid

  // 2. Compute deposit commitment
  component depositHasher = Poseidon(3);
  depositHasher.inputs[0] <== txHash;
  depositHasher.inputs[1] <== blockNumber;
  depositHasher.inputs[2] <== amount;
  depositCommitment <== depositHasher.out;

  // 3. Compute recipient hash
  component recipientHasher = Poseidon(1);
  recipientHasher.inputs[0] <== recipientAddress; // From private input
  recipientHash <== recipientHasher.out;

  // 4. Generate nonce (prevents double-spend)
  nonce <== Poseidon([txHash, blockNumber]);
}
```

Proof Generation Flow:

1. Backend listener detects deposit on Theta
2. Fetches block header and Merkle proof for transaction
3. Calls `snarkjs.groth16.fullProve()` with private inputs
4. Generates proof (192 bytes) + public signals
5. Submits proof to Persistence ZK verifier contract

Proof Verification (On-Chain):

The Persistence chain runs a ZK verifier contract (CosmWasm) that:

1. Verifies the ZK proof using the Groth16 verification key
2. Checks that the Merkle root matches a recent Theta block header (via IBC light client)
3. Verifies that the nonce hasn't been used before (prevents replay attacks)
4. If all checks pass, mints ibcTFUEL

Implementation Details:

- **Library:** `snarkjs` (JavaScript) for proof generation
 - **Curve:** BN254 (alt_bn128) for efficient pairing-based cryptography
 - **Proof size:** 192 bytes (2 G1 points + 1 G2 point)
 - **Verification time:** ~50ms (constant time, regardless of circuit complexity)
 - **Trusted setup:** Uses Perpetual Powers of Tau ceremony (community-audited)
-

3.3 IBC Integration & ibcTFUEL Minting

Once the ZK proof is verified on Persistence, **ibcTFUEL** is minted and transferred via IBC (Inter-Blockchain Communication) protocol.

IBC Channel-190

XFUEL uses IBC channel-190 to connect Persistence (as the hub) with other Cosmos chains:

Channel Configuration:

- **Source chain:** Persistence (core-1)
- **Destination chains:** Osmosis, Stride, Cosmos Hub
- **Port:** `transfer` (standard IBC transfer port)
- **Version:** ICS-20 (fungible token transfer)

IBC Transfer Flow:

```
// Simplified from backend/ibc/ibc-transfer.ts (lines 53-120)
async function transferViaIbc(
  amount: string,
  recipientAddress: string,
  memo?: string
) {
  const senderAddress = (await ibcWallet.getAccounts())[0].address

  const amountInDenom = {
    denom: IBC_CONFIG.ibc.tfuelIbcDenom, // 'ibc/TFUEL_HASH'
    amount: amount
  }

  const timeoutTimestamp = BigInt(Date.now() + 600_000) * 1_000_000n // 10 min

  const result = await ibcClient.sendIbcTokens(
    senderAddress,
    recipientAddress,
    amountInDenom,
    'transfer', // port
    IBC_CONFIG.persistence.ibcChannel, // channel-190
    undefined, // timeoutHeight
    timeoutTimestamp,
    IBC_CONFIG.ibc.gasLimit,
    memo
  )

  if (result.code !== 0) {
    throw new Error(`IBC transfer failed: ${result.rawLog}`)
  }

  return result.transactionHash
}
```

ibcTFUEL Token Mechanics

Supply:

- **Minting:** Only via ZK proof verification (1:1 with deposited TFUEL)
- **Burning:** When users bridge ibcTFUEL back to Theta (proof of burn required)
- **Peg mechanism:** TFUEL is locked in a Theta contract; ibcTFUEL is minted on Persistence. Burning ibcTFUEL unlocks TFUEL on Theta.

Use Cases:

1. **Swap to LSTs:** ibcTFUEL → stkTIA/stkATOM on Dexter DEX
2. **Liquidity provision:** Provide ibcTFUEL-USDC LP on Osmosis
3. **Collateral:** Use ibcTFUEL as collateral in Cosmos lending protocols (future)

Security:

- **No admin minting:** Only ZK verifier can mint ibcTFUEL
- **Nonce tracking:** Prevents double-minting from same Theta transaction
- **IBC light client:** Verifies Theta block headers to ensure proof validity

3.4 Smart Contract Layer

XFUEL's smart contracts are deployed on both Theta (EVM) and Persistence (CosmWasm).

Theta Contracts (Solidity)

XFUELRouter.sol:

- Handles TFUEL deposits and locks
- Emits `DepositReceived` event for backend listener
- Manages user withdrawal requests (when bridging back from Cosmos)
- Integrated with fee collection for protocol revenue

```
// Simplified interface
contract XFUELRouter {
    event DepositReceived(
        address indexed user,
        uint256 amount,
        bytes32 recipientHash, // Hash of Cosmos address for privacy
        uint256 nonce
    );

    function deposit(bytes32 recipientHash) external payable {
        require(msg.value >= MIN_DEPOSIT, "Amount too low");
        emit DepositReceived(msg.sender, msg.value, recipientHash, nextNonce++);
    }

    function withdraw(
        uint256 amount,
        bytes calldata zkProof, // Proof of ibcTFUEL burn on Persistence
        bytes32[] calldata publicSignals
    ) external {
        require(verifyBurnProof(zkProof, publicSignals), "Invalid proof");
        payable(msg.sender).transfer(amount);
    }
}
```

RevenueSplitter.sol:

- Collects swap fees, lottery rake, yield cuts
- Splits revenue: 90% to veXF holders, 10% to Innovation Treasury
- Triggers buyback & burn (25% of revenue → buy XF from DEX → burn)

```

contract RevenueSplitter {
    uint256 constant VEXF_SHARE = 9000; // 90%
    uint256 constant TREASURY_SHARE = 1000; // 10%
    uint256 constant BUYBACK_SHARE = 2500; // 25% of total

    function splitRevenue(uint256 totalRevenue) external {
        uint256 veXFAmount = (totalRevenue * VEXF_SHARE) / 10000;
        uint256 treasuryAmount = (totalRevenue * TREASURY_SHARE) / 10000;

        // Sub-split veXF share
        uint256 directYield = (veXFAmount * 5000) / 9000; // 50% of veXF share
        uint256 buybackAmount = (veXFAmount * 2500) / 9000; // 25% of veXF share
        uint256 rXFMintAmount = (veXFAmount * 1500) / 9000; // 15% of veXF share

        veXF.distributeYield(directYield);
        buybackBurner.execute(buybackAmount);
        rXF.mint(rXFMintAmount);
        innovationTreasury.receive(treasuryAmount);
    }
}

```

Persistence Contracts (CosmWasm)

ZKVerifier (zk_verifier.wasm):

- Verifies Groth16 ZK proofs on-chain
- Checks Merkle root against IBC light client's view of Theta
- Tracks nonces to prevent replay attacks
- Mints ibcTFUEL upon successful verification

```
// Simplified CosmWasm contract (Rust)
#[entry_point]
pub fn execute(
    deps: DepsMut,
    env: Env,
    info: MessageInfo,
    msg: ExecuteMsg,
) -> Result<Response, ContractError> {
    match msg {
        ExecuteMsg::VerifyAndMint {
            proof,
            public_signals,
        } => {
            // 1. Verify ZK proof
            let is_valid = groth16_verify(&proof, &public_signals, &VERIFYING_KEY)?;
            ensure!(is_valid, ContractError::InvalidProof);

            // 2. Check nonce hasn't been used
            let nonce = public_signals.nonce;
            ensure!(!NONCES.has(deps.storage, nonce), ContractError::NonceUsed);
            NONCES.save(deps.storage, nonce, &true)?;

            // 3. Verify Merkle root via IBC light client
            let merkle_root = public_signals.merkle_root;
            let is_valid_root = ibc_light_client::verify_theta_block(merkle_root)?;
            ensure!(is_valid_root, ContractError::InvalidMerkleRoot);

            // 4. Mint ibcTFUEL
            let amount = public_signals.amount;
            let recipient = decode_recipient_hash(public_signals.recipient_hash)?;
            mint_ibc_tfuel(deps, recipient, amount)?;

            Ok(Response::new()
                .add_attribute("action", "verify_and_mint")
                .add_attribute("amount", amount.to_string()))
        }
    }
}
```

ibcTFUEL Token (cw20_ibc_tfuel.wasm):

- CW20 token standard (Cosmos equivalent of ERC20)
- Minting restricted to ZK verifier contract
- Burnable (for bridging back to Theta)

3.5 Yield Optimization Engine

Once ibcTFUEL is minted, the **Yield Optimizer** automatically swaps it for the highest-yielding LST.

Supported LSTs & Strategies

LST	APY	Auto-Compounding	Rebalancing	Risk Tier
stkTIA	38.2%	Yes (daily)	Weekly	Medium
stkATOM	32.5%	Yes (daily)	Weekly	Low
stkXPRT	30.1%	Yes (daily)	Weekly	Medium
pSTAKE BTC	28.0%	Yes (daily)	Weekly	High

Rebalancing Logic:

```
// Simplified from backend/yield-optimizer.ts (conceptual)
async function optimizeYield(amount: string, userPreferences: YieldPrefs) {
  // Fetch real-time APYs from oracles
  const apys = await fetchLSTAPYs()

  // Apply user risk tolerance filter
  const eligibleLSTs = apys.filter(lst => lst.riskTier <= userPreferences.maxRisk)

  // Sort by APY (descending)
  eligibleLSTs.sort((a, b) => b.apy - a.apy)

  // Select top LST
  const targetLST = eligibleLSTs[0]

  // Execute swap on Dexter DEX
  const swapResult = await dexterClient.swap({
    offerAsset: 'ibcTFUEL',
    askAsset: targetLST.denom,
    amount: amount,
    slippage: 0.01 // 1% max slippage
  })

  // Stake on pStake (if applicable)
  if (targetLST.requiresStaking) {
    await pStakeClient.stake({
      amount: swapResult.receivedAmount,
      validator: targetLST.validatorAddress
    })
  }

  return swapResult
}
```

Weekly Rebalancing:

The protocol rebalances user positions weekly based on APY changes:

- If a new LST offers >2% higher APY, trigger rebalancing
- Gas costs are covered by protocol revenue (no user intervention)
- Users can opt out of rebalancing via governance settings

4. ibcTFUEL Tokenomics

4.1 Supply Mechanics

Total Supply: Dynamic (1:1 backed by locked TFUEL on Theta)

Metric	Value
Initial Supply	0 (minted on demand)
Max Supply	Unlimited (capped by TFUEL deposits)
Minting	Via ZK proof verification only
Burning	Via withdrawal request to Theta

4.2 Peg Stability

Mechanism: Algorithmic peg maintained via arbitrage:

1. **Premium ($\text{ibcTFUEL} > \text{TFUEL}$):** Arbitrageurs deposit TFUEL on Theta, receive ibcTFUEL, sell on Cosmos DEX for profit
2. **Discount ($\text{ibcTFUEL} < \text{TFUEL}$):** Arbitrageurs buy ibcTFUEL on Cosmos DEX, bridge to Theta, withdraw TFUEL

Peg Protection:

- **Emergency circuit breaker:** If ibcTFUEL deviates $>5\%$ from TFUEL for >24 hours, new mints are paused
- **Redemption guarantee:** Users can always redeem ibcTFUEL 1:1 for TFUEL (minus gas costs)

4.3 Yield Distribution (for LST Holders)

Users who swap ibcTFUEL for LSTs earn:

- **Base LST yield:** 30-38% APY (auto-compounded)
- **XF rewards:** 5% APY in XF tokens (protocol incentives)
- **Early adopter bonus:** 2× XF rewards for first 90 days

Example:

- User deposits 1,000 TFUEL
- Receives 1,000 ibcTFUEL
- Swaps for 1,000 stkTIA (38.2% APY)
- After 1 year:
 - stkTIA value: 1,382 stkTIA (~\$1,520 at \$1.10/stkTIA)
 - XF rewards: 50 XF tokens (~\$100 at \$2/XF)
 - **Total yield:** ~52% APY

5. XF Governance Token Tokenomics

5.1 Token Distribution

Total Supply: 100,000,000 XF (fixed, no inflation)

Allocation	%	XF (M)	Vesting	Use Case
Liquidity + Deflation Engine	45%	45	20% TGE, 80% via emissions	Initial DEX liquidity; buyback & burn sink
Community Flywheel & Real-Yield	30%	30	Earned via TVL milestones	Staking rewards, LP incentives, rXF minting
Perpetual Innovation Treasury	10%	10	25% TGE, 75% via revenue	Grants, acquisitions, experiments
Founder & Core Contributors	10%	10	1-year cliff, 4-year linear	Team incentives (performance-based cliffs)
Early Strategic Believers	5%	5	100% as rXF day 1	Soulbound receipts (12-month lock, then redeemable)

5.2 Revenue Flow

All protocol revenue (swap fees, lottery rake, yield cuts) flows into `RevenueSplitter.sol`:

```
Total Revenue (100%)
├── 90% → veXF Holders
│   ├── 50% → Direct USDC yield (claimable)
│   ├── 25% → Buyback XF from DEX → Burn (deflationary)
│   └── 15% → Mint rXF (revenue-backed receipts)
├── 10% → Innovation Treasury
│   ├── 40% → Builder Vault (micro-grants)
│   ├── 35% → Acquisition Vault (buy protocols)
│   └── 25% → Moonshot Vault (experiments → 50% airdrop to veXF/rXF)
```

5.3 veXF (Vote-Escrowed XF)

Lock Mechanics:

- Lock XF for 1-4 years → receive veXF (non-transferable)
- **Voting power:** Linear multiplier (1× at 1 year, 4× at 4 years)
- **Yield boost:** Longer locks receive proportionally more revenue share

Additional Multipliers:

1. **rXF Lock Boost:** Lock rXF for 365 days → +4× veXF voting power (additive)
2. **Theta Pulse Multiplier:** Prove Edge Node earnings via cryptographic signature → +1× to +3× permanent multiplier (based on earnings tier)

Max veXF Multiplier: 4× (base lock) + 4× (rXF) + 3× (Theta Pulse) = **11×**

5.4 rXF (Revenue-Backed Receipts)

Concept: Soulbound NFTs minted from protocol revenue (ERC721 with transfer disabled)

Minting:

- 15% of protocol revenue is used to mint rXF
- Mint price: Floor price of XF on DEX at mint time
- **Example:** If \$1,500 in revenue is allocated and XF = \$2, then 750 rXF are minted

Benefits:

- **4× veXF voting multiplier** when locked for 365 days
- **Priority airdrops** for spin-out tokens from Moonshot Vault
- **Revenue share:** rXF holders receive pro-rata share of future revenue (on top of veXF yield)

Lock Period:

- **Early Believers:** Receive rXF day 1, locked for 12 months (then redeemable 1:1 for XF)
- **Community mints:** Locked for 365 days for governance boost (optional)

5.5 Theta Pulse Proof Staking

Purpose: Prove Theta Edge Node earnings → permanent veXF multiplier

How it works:

1. User signs a message with their Theta Edge Node wallet: "I own node X, earned Y TFUEL"
2. Backend validates signature against Theta's TPulse (public Edge Node registry)
3. If valid, user receives permanent multiplier based on earnings tier:
 - **Tier 1:** 100-1,000 TFUEL/month → +1× multiplier
 - **Tier 2:** 1,000-10,000 TFUEL/month → +2× multiplier
 - **Tier 3:** >10,000 TFUEL/month → +3× multiplier

Smart Contract Interface:

```
interface IThetaPulseProof {
    function submitProof(
        bytes calldata signature,
        uint256 earningsAmount,
        uint256 timestamp
    ) external returns (uint256 multiplier);

    function getMultiplier(address account) external view returns (uint256);
}
```

5.6 Cybernetic Fee Switch

Governance-Controlled Fee Modes:

Mode	Swap Fee	Lottery Rake	Yield Cut	Purpose
Growth Mode	0.05%	2%	3%	Attract TVL, bootstrap liquidity
Balanced Mode	0.3%	5%	5%	Default (current)
Extraction Mode	0.5%	10%	8%	Maximize revenue for veXF holders

Voting:

- veXF holders vote on fee mode (quadratic voting to prevent whale dominance)
- Proposal requires 20% quorum
- 48-hour voting period
- 24-hour timelock before execution

6. Security Model & Cryptographic Guarantees

6.1 ZK Proof Security

Threat Model:

- **Attacker goal:** Mint ibcTFUEL without depositing TFUEL on Theta

Defenses:

1. **ZK-SNARK soundness:** Computationally infeasible to forge proof (requires breaking BN254 elliptic curve)
2. **Merkle root verification:** Proof must reference a valid Theta block header (verified via IBC light client)
3. **Nonce tracking:** Each Theta transaction can only mint ibcTFUEL once
4. **Trusted setup:** Uses audited Powers of Tau ceremony (252 participants)

Security Assumptions:

- Groth16 ZK-SNARK is secure under discrete log and knowledge of exponent assumptions
- IBC light client correctly syncs Theta block headers
- Backend prover is honest (can be decentralized via incentivized relayers in future)

6.2 IBC Security

Threat Model:

- **Attacker goal:** Double-spend ibcTFUEL or manipulate cross-chain transfers

Defenses:

1. **IBC light client:** Each chain maintains light client of counterparty (verifies block headers)
2. **Packet commitment:** All IBC transfers are cryptographically committed in Merkle tree
3. **Timeout & acknowledgment:** Failed transfers are automatically rolled back

Security Assumptions:

- Persistence validators are honest (51% assumption)
- IBC relayers are live (liveness failure → timeouts, not security failure)

6.3 Smart Contract Security

Audits:

- **Theta contracts:** Audited by [Audit Firm TBD]
- **Persistence contracts:** Audited by [Audit Firm TBD]

Key Protections:

- **Reentrancy guards:** All external calls use `ReentrancyGuard`
 - **Access control:** Only authorized addresses can mint ibcTFUEL
 - **Upgradability:** Contracts use UUPS proxy pattern (veXF-governed upgrades)
 - **Circuit breakers:** Emergency pause if >5% peg deviation or exploit detected
-

7. Risks & Mitigations

7.1 Technical Risks

Risk	Severity	Probability	Impact	Mitigation
ZK proof forgery	 Critical	Very Low	Total loss of funds	- Audited ZK circuit - Trusted setup ceremony - Merkle root verification via IBC light client - Nonce tracking prevents replay
IBC relayer failure	 Medium	Medium	Delayed transactions	- Multiple relayer instances (decentralized) - Automatic retry logic - Timeout refunds if delivery fails
Smart contract exploit	 Critical	Low	Loss of user funds	- Multi-firm audits - Bug bounty program (\$500K) - Emergency pause function - Upgradeable contracts with 48h timelock
Oracle manipulation	 Medium	Low	Incorrect APY data → suboptimal yields	- Use multiple oracles (Chainlink, Band, Pyth) - TWAP (time-weighted avg price) - Sanity checks (reject >10% deviation)
Backend prover downtime	 Low	Medium	Delayed minting (no loss of funds)	- High availability (99.9% SLA) - Auto-restart on failure - Decentralized prover network (Phase 2)

Critical Path Analysis:

The most critical failure mode is a **ZK proof forgery** that allows minting ibcTFUEL without locking TFUEL. This is prevented by:

1. Merkle root verification (proof must reference a valid Theta block)
2. Nonce uniqueness (each deposit can only be proven once)
3. Backend monitoring (alerts if ibcTFUEL supply exceeds locked TFUEL)

7.2 Economic Risks

Risk	Severity	Probability	Impact	Mitigation
ibcTFUEL depeg	 Medium	Medium	Arbitrage losses, user panic	- Emergency circuit breaker (pause minting if >5% depeg) - Incentivized arbitrage pool - Redemption guarantee (1:1 burn for TFUEL)
LST smart contract failure	 Critical	Very Low	Loss of staked funds	- Only integrate audited LSTs (Stride, pStake) - Diversification across multiple LSTs - Insurance fund (8% of TVL in TreasuryILBackstop)
TFUEL price crash	 Medium	Medium	Lower APY in USD terms	- Diversify into stablecoins (USDC yield) - veXF holders receive stablecoin revenue - No protocol liquidations (non-collateralized)
Whale governance attack	 Medium	Low	Malicious proposals pass	- Quadratic voting (sqrt of veXF balance) - 48h timelock on execution - Emergency veto by multisig (2/3) for first 6 months
Revenue share imbalance	 Low	Low	veXF holders or treasury underfunded	- Hardcoded revenue split (90/10) - Governance can adjust with 7-day timelock - Historical data dashboards for transparency

Stress Test Scenarios:

1. 50% TFUEL price drop:

- ibcTFUEL remains pegged (1:1 with TFUEL, not USD)
- LST yields remain stable (APY is in native tokens)
- XF token may suffer (reduced TVL → lower revenue → lower buyback)

2. IBC relayer outage (6 hours):

- Deposits accumulate on Theta (no loss)
- Proofs queued by backend
- Automatic processing resumes when relayer returns
- Max delay: 6 hours (no impact on funds safety)

3. Major LST (stkTIA) exploit:

- Emergency pause of stkTIA swaps
- Users can withdraw to ibcTFUEL (not affected)
- Insurance fund covers up to 8% of affected TVL
- Governance vote to redistribute remaining users to other LSTs

7.3 Regulatory Risks

Risk	Severity	Probability	Impact	Mitigation
Securities classification	 Medium	Medium (US), Low (global)	Potential enforcement action	- XF has governance utility (not just profit expectation) - Decentralized (no company controls protocol) - Legal opinion from [Firm TBD] - Restrict US users if needed (geofencing)
AML/KYC requirements	 Low	Low	Need to implement identity checks	- Currently permissionless (no custody) - Can add optional KYC for large deposits if required - Chainalysis monitoring for illicit funds
Sanctions compliance	 Low	Low	Blocked addresses	- Smart contract blacklist (OFAC addresses) - Backend checks for sanctioned entities - No control over on-chain LST transfers (non-custodial)

Compliance Strategy:

- Protocol is designed to be **maximally decentralized** (no admin keys after 6 months)
- veXF governance controls all parameters (fee rates, treasury allocations, etc.)
- Legal structuring as a DAO (no central entity responsible for user funds)

7.4 Operational Risks

Risk	Severity	Probability	Impact	Mitigation
Backend server compromise	 Critical	Low	Attacker could submit fake proofs	- Private keys stored in HSM (hardware security module) - No private keys give minting authority (only ZK verifier can mint) - Rate limiting on proof submission - Multi-region deployment with failover
Frontend phishing	 Medium	Medium	Users send funds to fake site	- HTTPS + SSL pinning - Publish official domain on GitHub/Twitter - WalletConnect verification - Browser extension warnings
Dependency vulnerabilities	 Medium	Medium	Exploit in snarkjs, ethers, etc.	- Automated Dependabot alerts - Pin exact versions (no ^ in package.json) - Monthly security updates - Audit dependencies with Snyk

8. Governance & Sustainability

8.1 Governance Framework

veXF Voting Power:

- 1 XF locked for 4 years = 4 veXF
- rXF locked for 365 days = 4× multiplier (16 veXF per XF)
- Theta Pulse proof = +1× to +3× multiplier (up to 48 veXF per XF)

Proposal Types:

Type	Quorum	Approval	Timelock	Examples
Parameter Change	10%	> 50%	24h	Adjust swap fees, yield cuts
Treasury Spend	20%	> 60%	48h	Grant \$50K to developer
Contract Upgrade	30%	> 66%	7 days	Upgrade RevenueSplitter logic
Emergency Action	5%	> 75%	0h	Pause protocol if exploit detected

Voting Mechanism:

- **Quadratic voting:** `votingPower = sqrt(veXF_balance)` (prevents whale dominance)
- **Delegation:** Users can delegate voting power to trusted community members
- **Veto:** Core multisig (2/3) can veto proposals for first 6 months (then auto-renounce)

8.2 Innovation Treasury Vaults

Vault Allocation (10% of all revenue):

1. Builder Vault (40% of treasury):

- Micro-grants up to \$5K
- No application process (permissionless, veXF voting)
- Focus: Open-source tools, integrations, SDKs

2. Acquisition Vault (35% of treasury):

- Buy revenue-generating protocols (DEXs, lending platforms)
- Due diligence by community (public audits)
- Acquired revenue flows back to veXF holders

3. Moonshot Vault (25% of treasury):

- Fund high-risk experiments (ZK rollups, AI agents)
- Successful projects spin out as separate tokens
- 50% of spin-out tokens airdropped to veXF/rXF holders

Example:

- Protocol earns \$100K in one month
- \$10K goes to Innovation Treasury
- \$4K → Builder Vault (8 grants × \$500 each)
- \$3.5K → Acquisition Vault (accumulating for larger purchase)
- \$2.5K → Moonshot Vault (funds 3-month ZK experiment)

8.3 Long-Term Sustainability

Revenue Projections:

Year	TVL	Monthly Volume	Revenue (0.3% fee)	Annual Revenue
1	\$5M	\$500K	\$1.5K	\$18K
2	\$20M	\$2M	\$6K	\$72K
3	\$50M	\$5M	\$15K	\$180K
5	\$200M	\$20M	\$60K	\$720K

Deflation Mechanics:

- 25% of revenue → buyback XF from DEX → burn
- **Year 1:** ~\$4,500 in buybacks → ~2,250 XF burned (at \$2/XF)
- **Year 5:** ~\$180K in buybacks → ~90K XF burned
- **10-year projection:** ~5-10% of supply burned (if growth continues)

Sustainability Triggers:

- If revenue < \$10K/month → reduce treasury allocation (keep veXF share at 90%)
 - If TVL > \$100M → increase insurance fund to 10% (from 8%)
 - If XF price > \$10 → accelerate buyback rate to 35% (from 25%)
-

9. Roadmap

Phase 1: Foundation (Q1 2025) COMPLETE

- ☒ Deploy XFUELRouter on Theta Mainnet
- ☒ Implement manual deposit flow (QR codes)
- ☒ Launch web app (xfuel.app)
- ☒ Deploy TipPool lottery contracts
- ☒ Integrate Theta Wallet (no extensions)

Phase 2: ZK Bridge (Q2 2025) IN PROGRESS

- ☐ Implement ZK proof system (Circom circuit)
- ☐ Deploy ZK verifier on Persistence
- ☐ Launch ibcTFUEL minting (testnet)
- ☐ IBC channel-190 integration
- ☐ Mainnet launch (limited beta)

Phase 3: Yield Automation (Q3 2025)

- ☐ Integrate Dexter DEX for LST swaps
- ☐ Implement pStake staking automation
- ☐ Deploy yield optimizer (stkTIA, stkATOM)
- ☐ Weekly rebalancing engine
- ☐ Mobile app (Expo + React Native)

Phase 4: Tokenomics Upgrade (Q4 2025)

- ☐ Deploy veXF governance contracts
- ☐ Launch rXF revenue receipts
- ☐ Implement RevenueSplitter (90/10 split)
- ☐ Deploy Innovation Treasury vaults
- ☐ Theta Pulse Proof integration

Phase 5: Decentralization (Q1 2026)

- ☐ Decentralized prover network (incentivized relayers)

- ☐ Transfer admin keys to veXF governance
- ☐ Multi-region backend deployment
- ☐ Bug bounty program launch (\$500K pool)
- ☐ Third-party audit publication

Future Research

- **ZK rollup:** Batch multiple deposits into single proof (100× gas savings)
 - **Privacy features:** Hide deposit amounts using homomorphic encryption
 - **AI yield optimizer:** Train ML model on historical APY data
 - **Cross-chain expansion:** Bridge to Ethereum, BNB Chain, Avalanche
-

10. Conclusion

XFUEL represents a paradigm shift in cross-chain DeFi infrastructure by combining:

1. **Zero-knowledge proofs** for trustless bridging
2. **IBC protocol** for secure cross-chain messaging
3. **Automated yield optimization** for maximum APY
4. **Innovative tokenomics** that align protocol growth with holder value

By eliminating custodial risk, wallet connection friction, and manual staking complexity, XFUEL lowers the barrier to cross-chain yield for Theta's 10,000+ Edge Node operators and the broader Cosmos community.

The core innovation: A ZK bridge that proves deposits on Theta without trusting centralized relayers, mints ibcTFUEL 1:1 on Persistence, and automatically swaps to the highest-yielding LST — all in under 4 seconds.

The long-term vision: A perpetual yield pumping station where protocol revenue funds buyback & burn (deflation), direct yield to veXF holders (real yield), and innovation treasury experiments (holder-owned R&D). Early believers, Theta Edge Node operators, and active governance participants receive permanent multipliers on rewards and voting power.

Live now at xfuel.app — the pumps are primed.

11. References & Appendices

A. Technical References

1. **Groth16 ZK-SNARKs:**

Jens Groth. "On the Size of Pairing-based Non-interactive Arguments." *EUROCRYPT 2016*.
<https://eprint.iacr.org/2016/260>

2. IBC Protocol:

Cosmos Network. "IBC Protocol Specification."

<https://github.com/cosmos/ibc>

3. Circom ZK Language:

iden3. "Circom Documentation."

<https://docs.circom.io/>

4. Powers of Tau Ceremony:

Perpetual Powers of Tau (252 participants).

<https://github.com/weijiekoh/perpetualpowersoftau>

B. Smart Contract Addresses

Theta Mainnet (Chain ID: 361):

- XFUELRouter: [To be deployed]
- RevenueSplitter: [To be deployed]
- TipPool: [Deployed at existing address]

Persistence Mainnet (core-1):

- ZKVerifier: [To be deployed]
- ibcTFUEL (CW20): [To be deployed]

C. Audit Reports

- **Theta Contracts:** [Audit Firm TBD] (Q2 2025)
- **Persistence Contracts:** [Audit Firm TBD] (Q2 2025)
- **ZK Circuit:** [Independent Cryptographer TBD] (Q2 2025)

D. Security Contact

- **Bug Bounty:** security@xfuel.app
- **Emergency Contact:** emergency@xfuel.app (24/7 monitoring)
- **PGP Key:** [To be published on GitHub]

E. Links

- **Website:** <https://xfuel.app>
- **GitHub:** <https://github.com/XFuel-Lab/xfuel-protocol>
- **Twitter:** @XFUEL
- **Discord:** [Community server TBD]
- **Docs:** <https://docs.xfuel.app>

F. Glossary

- **APY:** Annual Percentage Yield (compounded)

- **IBC:** Inter-Blockchain Communication (Cosmos protocol)
 - **LST:** Liquid Staking Token (e.g., stkTIA)
 - **rXF:** Revenue-backed soulbound receipt NFT
 - **TFUEL:** Native token of Theta Network
 - **veXF:** Vote-escrowed XF (governance token)
 - **ZK-SNARK:** Zero-Knowledge Succinct Non-Interactive Argument of Knowledge
-

Prepared by XFUEL Core Team

December 2025

Version: 2.0 (ZK Bridge Edition)

License: Creative Commons BY-NC-SA 4.0

Disclaimer

This whitepaper is for informational purposes only and does not constitute financial, legal, or investment advice. XFUEL is experimental software with inherent risks including smart contract vulnerabilities, market volatility, regulatory uncertainty, and potential loss of funds. The protocol is provided "as is" without warranties. Users assume all risks. The core team makes no guarantees of returns, security, or protocol performance. Regulatory treatment of crypto assets varies by jurisdiction; users are responsible for compliance with local laws. This document may contain forward-looking statements that are subject to change. Always do your own research and never invest more than you can afford to lose.

End of Whitepaper